

Laboratório 3: Dados relacionais e operações do tipo join

Ana Julia Sanches de Araujo

```
# Carregue os pacotes relevantes para este Laboratório

library(readr)
library(dplyr)
```

1. Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE; Para o arquivo de vôos: a. Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY; b. Leia apenas 1 milhão de linhas por vez; c. Remova vôos em que o aeroporto de destino comece com a letra 1; d. Remova registros em que existam pelo menos uma coluna faltante; e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino; f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
# Importa o arquivo inteiro sem restrições
airlines <- read_csv("airlines.csv")
```

Rows: 14 Columns: 2

```
-- Column specification -----
Delimiter: ","
chr (2): IATA_CODE, AIRLINE
```

- i Use `spec()` to retrieve the full column specification for this data.
- i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Importa os aeroportos filtrando as colunas na hora da leitura
airports <- read_csv("airports.csv",
  col_types = cols_only(
    IATA_CODE = col_character(),
    CITY = col_character(),
    STATE = col_character(),
    LATITUDE = col_double(),
    LONGITUDE = col_double()
  )
)
```

```
# Regras A e B: Lê o arquivo em lotes de 1 milhão e com apenas 2 colunas
# Criando a função que filtra e calcula estatísticas para cada lote
```

```
processa_lote <- function(df, pos) {
  df %>%
    filter(!startsWith(DESTINATION_AIRPORT, "1")) %>%
    filter(!is.na(DESTINATION_AIRPORT) & !is.na(ARRIVAL_DELAY)) %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma_dos_atrasos = sum(ARRIVAL_DELAY),
      quantidade_de_voos = n(),
      .groups = "drop"
    )
}

lotes_processados <- read_csv_chunked(
  file = "flights.csv",
  chunk_size = 1000000,
  callback = DataFrameCallback$new(processa_lote),
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)
```

```
# Regra F: Junta todos os lotes e calcula a média verdadeira
flights <- lotes_processados %>%
  # Agrupa novamente, pois o aeroporto "JFK" pode ter aparecido no lote 1 e no lote 2
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    soma_final = sum(soma_dos_atrasos),
    voos_finais = sum(quantidade_de_voos),
```

```

    .groups = "drop"
  ) %>%
  # Finalmente, cria a coluna de média dividindo o total pelo número de voos
  mutate(atraso_medio = soma_final / voos_finais) %>%
  # Joga fora as colunas de soma e mantém só o destino e a média
  select(DESTINATION_AIRPORT, atraso_medio)

```

2. Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa:

- a. Identifique a coluna que é a chave na tabela flights;
- b. Identifique a coluna que é a chave na tabela airports; c. Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports?
- c. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
- d. Armazene a tabela resultante no objeto flights.

```

# c. Aeroportos em flights ausentes em airports
aeroportos_ausentes <- anti_join(
  x = flights,
  y = airports,
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)

```

```

# Exibe o resultado na tela
aeroportos_ausentes

```

```

# A tibble: 0 x 2
#   i 2 variables: DESTINATION_AIRPORT <chr>, atraso_medio <dbl>

```

```

# d e e. Combinando as tabelas e salvando no objeto flights
flights <- flights %>%
  left_join(
    airports,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) %>%
  # Mantendo apenas as colunas que importam para deixar a tabela limpa
  select(DESTINATION_AIRPORT, atraso_medio, CITY, STATE, LATITUDE, LONGITUDE)

# Exibe as primeiras linhas para conferir se deu certo
head(flights)

```

```
# A tibble: 6 x 6
  DESTINATION_AIRPORT atraso_medio CITY      STATE LATITUDE LONGITUDE
  <chr>                <dbl> <chr>    <chr>    <dbl>    <dbl>
1 ABE                  5.80 Allentown PA        40.7     -75.4
2 ABI                  4.08 Abilene   TX        32.4     -99.7
3 ABQ                  5.61 Albuquerque NM        35.0    -107.
4 ABR                 -3.39 Aberdeen SD        45.4     -98.4
5 ABY                  8.68 Albany   GA        31.5     -84.2
6 ACK                  4.91 Nantucket MA        41.3     -70.1
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
# 3. Contagem de aeroportos por estado em ordem decrescente
contagem_estados <- airports %>%
  group_by(STATE) %>%
  summarise(
    num_aeroportos = n(),
    .groups = "drop"
  ) %>%
  # arrange() ordena a tabela; desc() garante que seja do maior para o menor
  arrange(desc(num_aeroportos))

# Exibe o resultado final
contagem_estados
```

```
# A tibble: 54 x 2
  STATE num_aeroportos
  <chr>      <int>
1 TX          24
2 CA          22
3 AK          19
4 FL          17
5 MI          15
6 NY          14
7 CO          10
8 MN           8
9 MT           8
10 NC           8
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

- a. Carregue o pacote leaflet;
- b. Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico;
- c. Armazene o grafico em b) numa variável chamada this;
- d. Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

```
# a. Carregue o pacote leaflet
library(leaflet)

# b e c. Combine os comandos e armazene na variável 'this'
this <- leaflet(data = flights) %>%
  addTiles() %>% # Desenha o mapa base (ruas, continentes, etc)
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    # Um bônus: adicionando o nome e o atraso quando clicar no balãozinho!
    popup = ~paste(DESTINATION_AIRPORT, "<br>Atraso Médio:", round(atraso_medio, 2))
  )
```

Warning in validateCoords(lng, lat, funcName): Data contains 3 rows with either missing or invalid lat/lon values and will be ignored

```
# this
```

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-31 21:17:55 -03"
```